

OPERATOR GUIDE

STEP-BY-STEP

NAUTOBOT · GITLAB · ACI

Network Platform Engineering Lab Operator User Guide

How to enter Nautobot data, transact with GitLab, run diffs and verification, and use every MCP-exposed tool (OPA, Terraform, Ansible, MinIO) before pushing configuration to Cisco ACI

Guide date: 1 September 2026

Platform: C:\MCP\INFRA-AUTOMATION-PLATFORM

Audience: engineers operating the lab day-to-day — entering intent in Nautobot, reviewing/approving GitLab pipelines, and validating the result before and after it reaches Cisco ACI.

Scope: every stage of the Execution Framework lifecycle — Intent (Nautobot) → Validation → Policy (OPA) → Approval → Execution (Terraform/Ansible) → Verification (pyATS) → Knowledge Capture (MinIO) — plus the MCP Server tools an AI agent can call on your behalf.

Table of Contents

1. Lab architecture at a glance
2. Component responsibility matrix — correcting a common misconception
3. End-to-end data flow and transactions
4. GitLab's full role beyond version control, and its relationship to GitHub
5. AI, MCP, and the "AI reasons, platform executes" boundary
6. Before you start — prerequisites and credentials
7. Step 1 — Entering data in Nautobot
8. Step 2 — Using the MCP Server tools (the AI-agent path)
9. Step 3 — Generating the NetAsCode YAML from Nautobot
10. Step 4 — Reviewing diffs before anything is applied
11. Step 5 — Schema validation
12. Step 6 — OPA policy enforcement
13. Step 7 — Working with GitLab (project, variables, triggers, approvals)
14. Step 8 — Terraform plan and apply against Cisco ACI
15. Step 9 — Ansible Day-2 configuration
16. Step 10 — pyATS independent verification
17. Step 11 — MinIO knowledge capture and audit trail
18. End-to-end worked example (one tenant, start to finish)
19. Command cheat-sheet
20. Future integration — Jira as a ticketing front-door for Day-2 operations
21. Instructing Nautobot via a Low-Level Design (LLD) document
22. Future enhancement — a LangGraph reasoning agent
23. Troubleshooting and operational hazards

Component	Purpose	Where it runs
Nautobot	Source of Truth for Tenants/VRFs/Prefixes/VLANs — your data entry point	<code>http://localhost:8081</code> (isolated lab instance) or <code>:8080</code> (full platform stack)
MCP Server	AI-agent-facing tool layer; writes structured objects to Nautobot on your behalf	<code>http://localhost:8071/mcp</code>
GitLab CE + Runner	Execution engine — runs every pipeline stage below in order	<code>http://localhost:8929</code>
OPA	Policy engine — evaluates generated YAML against Rego rules	<code>http://localhost:8181</code>
HashiCorp Vault	Stores APIC/GitLab/MinIO credentials read by Terraform, Ansible, pyATS, and the generator	<code>http://localhost:8200</code>
Terraform (<code>CiscoDevNet/aci</code>)	Applies the NetAsCode YAML's desired state to the real APIC	Runs inside a GitLab CI job, or locally from <code>platform/terraform/aci/</code>
Ansible (<code>cisco.aci</code>)	Day-2 configuration (e.g. EPG creation) after Terraform apply	<code>platform/ansible/aci/</code>
pyATS	Independent, read-only verification against the live APIC	<code>tests/pyats/aci/</code>
MinIO	Durable JSONL knowledge-capture log of every deployment	<code>http://localhost:9000</code> , bucket <code>knowledge-capture</code>

What's running in your current isolated lab today: the isolated Nautobot instance (`http://localhost:8081`) and an isolated GitLab instance (`http://localhost:8929`) are up. OPA, Vault, MinIO, and the GitLab Runner are *not yet part of that lightweight profile* — this guide shows you exactly how each one is used so you can add them (from `docker/other-containers/` and `docker/vault/`) when you are ready to exercise the full pipeline end-to-end, and also shows the equivalent manual commands you can run right now without them.

2. Component Responsibility Matrix — Correcting a Common Misconception

Nautobot does not run Terraform or Ansible, and it never talks to Cisco ACI directly. This is the single most important architectural fact in this lab, so it is stated plainly before anything else: Nautobot is a Source of Truth database with a webhook, not an execution engine. Per `knowledge/adr/ADR-019-Three-Truths-Principle.md`: *"Nautobot owns desired network state... this is Truth #2 (Desired State), not Truth #1 (Business Intent)."* Execution is a completely separate concern, owned end-to-end by GitLab.

Component	IS responsible for	IS NOT responsible for
Nautobot	Storing Tenants/VRFs/Prefixes/VLANs as the network Source of Truth; firing a webhook when a Tenant changes; storing post-execution status (<code>validation_status</code> , <code>last_pipeline_id</code>) written back by the pipeline	Running Terraform or Ansible; deciding policy; talking to the APIC; sequencing any part of the pipeline
Generator (<code>generate_aci.py</code>)	Deterministically translating Nautobot's live state into the ACI-specific NetAsCode YAML artifact	Evaluating policy; touching ACI; deciding whether a change is approved
GitLab CE + Runner	Owning execution, approvals, retries, concurrency locking, artifacts, logs, and pipeline history for every stage — see §4 for the full list	Deciding what to build (that is Nautobot + the generator's job); evaluating policy itself (delegated to OPA); verifying the result itself (delegated to pyATS)
OPA	Evaluating the generated YAML against Rego policy and returning an allow/deny decision, fail-closed if unreachable	Touching ACI; running Terraform; storing state
Terraform (<code>CiscoDevNet/aci</code>)	Applying the already-planned, already-approved desired state to the real APIC	Deciding what the desired state should be — it only ever executes what the approved YAML/plan already says
Ansible (<code>cisco.aci</code>)	Day-2 configuration tasks and a secondary tenant-existence sanity check after Terraform apply	Primary provisioning (that is Terraform's job); running before Terraform
pyATS	Independent, read-only REST verification against the live APIC, deliberately bypassing Terraform's own state	Applying any change — it is verification-only
MCP Server	Exposing thin, per-tool business operations (<code>create_tenant</code> , etc.) that write structured objects straight to Nautobot, and reading status back	Orchestrating the pipeline itself — per ADR-016, it "must never become another orchestration engine"
MinIO	Durable JSONL knowledge-capture log of every deployment	Being any kind of configuration source
HashiCorp Vault	Storing credentials read by Terraform, Ansible, pyATS, and the generator	Any decision-making role

So, concretely: Nautobot is never "acting as an IaC tool" and never "runs" Terraform or Ansible. The actual chain is *Nautobot (data) → generator (translation) → GitLab (execution, running Terraform/Ansible as CI jobs) → ACI*. Nautobot's only active behavior beyond storing data is firing one outbound webhook when a Tenant changes.

3. End-to-End Data Flow and Transactions

Every hop below is a real network call with a specific protocol and endpoint — nothing is implicit or in-process across these boundaries.

#	From → To	Protocol / call	What is exchanged
1	Engineer / AI Agent → Nautobot (via MCP) or Nautobot UI	POST /api/tenancy/tenants/ etc., Token auth	A structured object write — Tenant/VRF/Prefix/VLAN
2	Nautobot → GitLab	Nautobot's native Webhook (Jinja2 body template) → POST /api/v4/projects/:id/trigger/pipeline	A pure "something changed" signal (trigger token + ref) — no object payload is sent; the pipeline re-reads Nautobot itself
3	GitLab job → Nautobot	generate_aci.py 's GraphQL client → POST /api/graphql/	Full current Tenant/VRF/Prefix/VLAN state, including _custom_field_data
4	GitLab job → Git repository	git commit + git push (using GIT_PUSH_TOKEN)	The regenerated, deterministic tenants.yaml
5	GitLab job → OPA	policy_check.py → POST {OPA_URL}/v1/data/platform/cisco_aci/decision	The parsed YAML as input.domain_intent ; response {allow, reasons}
6	GitLab job → APIC	Terraform's CiscoDevNet/aci provider, HTTPS REST	The planned Tenant/VRF/BD/AP/EPG/Contract/L3Out objects
7	GitLab job → APIC	Ansible's cisco.aci collection, HTTPS REST (ansible_connection: local , no SSH)	Day-2 EPG creation; a query-only tenant existence check
8	GitLab job → APIC	pyATS' generic rest.connector , HTTPS REST GET api/class/fvTenant.json	Read-only confirmation that expected tenants exist — independent of Terraform's own state
9	GitLab job → Nautobot	write_results.py → PATCH /api/tenancy/tenants/{id}/	validation_status , last_pipeline_id , last_pipeline_url , last_validated_at
10	GitLab job → MinIO	capture_knowledge.py → boto3 S3 put_object , SigV4	One appended JSONL record — pipeline ID, commit SHA, operator, domain, status
11	Engineer / AI Agent → Nautobot + GitLab	show_status MCP tool → Nautobot REST read + GitLab GET /api/v4/projects/:id/pipelines	Merged stored status + live pipeline status

Notice that step 2 carries no object data at all — the webhook is a doorbell, not a payload courier. Everything the pipeline needs, it re-reads live from Nautobot in step 3. This is deliberate: it means the pipeline always acts on Nautobot's *current* state, never on a possibly-stale webhook body.

4. GitLab's Full Role Beyond Version Control, and Its Relationship to GitHub

4.1 What GitLab actually does in this lab

Per `knowledge/architecture/Platform-v2-Reference-Architecture.md`: "GitLab CE + GitLab Runner own execution, approvals, retries, concurrency, resource locking, artifacts, logs, and pipeline history." Concretely, that means:

Responsibility	GitLab-native mechanism used
Executing every stage in order	CI/CD pipeline stages + <code>needs:</code> job dependencies
Storing the version-controlled desired-state artifact	The Git repository itself — <code>platform/netascode/aci/tenants.yaml</code> is a normal tracked file
Human approval before any real change	Manual job (<code>when: manual</code>) gated by the <code>main</code> branch's protected-branch rule
Preventing two concurrent applies from corrupting Terraform state	<code>resource_group:</code> <code>"terraform-\${POLICY_DOMAIN}-\${CI_ENVIRONMENT_NAME}"</code>
Retrying transient failures	<code>retry:</code> keyword on CI jobs
Preserving evidence (plans, pyATS results, logs)	<code>artifacts:</code> , including <code>when: always</code> so failures still leave evidence
Being triggered by Nautobot without a human in the loop	A dedicated <code>Ci::Trigger</code> pipeline trigger token (\$13.3)
Complete execution history	Pipeline list, job logs, and commit history — all queryable via the GitLab REST API

In short: GitLab is **the execution engine**, not just a place to store YAML. Version control is one of its jobs — orchestrating the entire Execution Framework lifecycle (Intent → Validation → Policy → Approval → Execution → Verification → Knowledge Capture, per `knowledge/architecture/Execution-Framework.md`) is the rest of it.

4.2 How does GitLab communicate with GitHub?

Today, it does not. This repository's own source code is version-controlled on GitHub (`git remote -v` confirms `origin = https://github.com/panosnaz/infra-automation-platform.git`) — that is where humans collaborate on and review the platform's own code, ADRs, and pipeline definitions. The **self-hosted GitLab CE instance running inside this lab** is a completely separate system: it is the operational execution engine that actually runs pipelines against Nautobot/APIC. There is no GitHub Actions workflow, no push-mirror, and no webhook bridge configured between the two anywhere in this repository — confirmed by a direct search of tracked files.

These two systems serve two different purposes and are not meant to be the same thing:

	GitHub (github.com/panosnaz/infra-automation-platform)	GitLab CE (self-hosted, in this lab)
Purpose	Where the platform's own source code is developed and reviewed	Where the generated NetAsCode YAML lives and where pipelines actually execute against live infrastructure
Contains	This entire repository — docs, ADRs, MCP server code, pipeline definitions, Terraform/Ansible modules	A copy of the same <code>pipelines//platform/</code> content, pushed in once, then modified in-place by <code>generate_nac</code> 's auto-commits
Runs CI?	No — <code>.github/</code> is an intentionally empty placeholder; GitHub Actions was considered and explicitly not built (ADR-016 replaced that plan with GitLab CI)	Yes — this is the only place any pipeline actually runs

If you want to connect them in the future (not implemented today)

Three honest options, in increasing order of complexity — pick the simplest one that solves a real problem you actually have, rather than adding integration for its own sake:

1. **Keep them separate (recommended default).** One team, one lab — the extra synchronization complexity of connecting two Git remotes usually costs more than it saves.
2. **One-way push mirror, GitHub → GitLab.** GitLab natively supports "Pull mirroring" from an external Git repository (**Settings** → **Repository** → **Mirroring repositories**) — GitHub stays the single place engineers push code, GitLab automatically pulls the latest `pipelines//platform/` content on a schedule.
3. **GitHub Actions calling GitLab's trigger API.** A workflow in `.github/workflows/` could call the exact same `POST /api/v4/projects/:id/trigger/pipeline` endpoint documented in §13.3 whenever code is pushed to GitHub — useful only if you want GitHub pushes (not just Nautobot changes) to also trigger a GitLab pipeline run.

5. AI, MCP, and the "AI Reasons, Platform Executes" Boundary

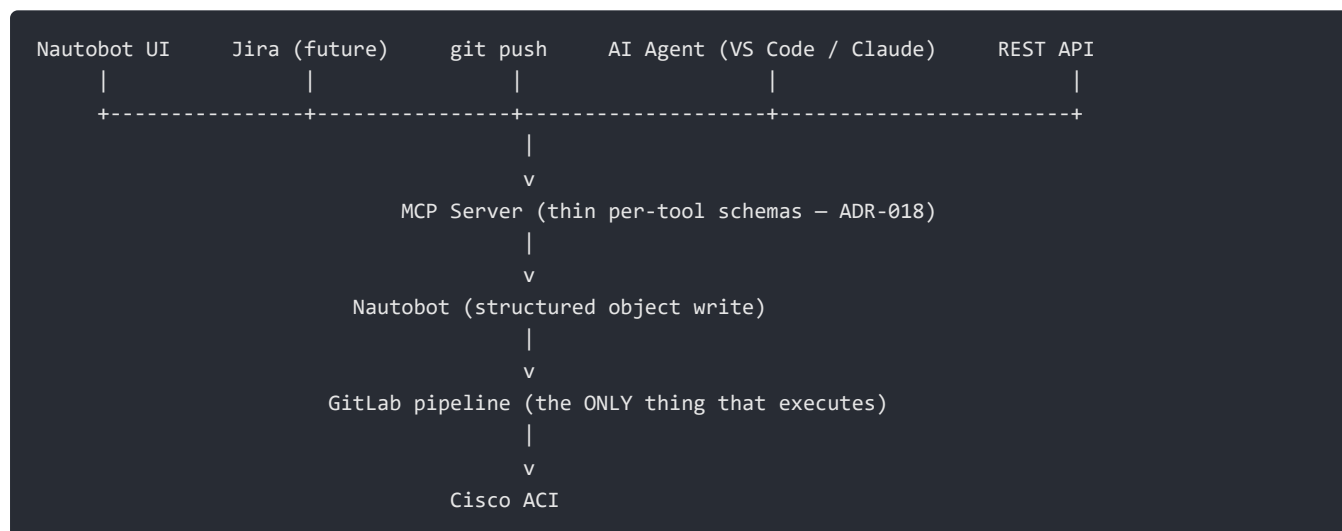
This boundary is a first-class, named architectural decision, not an incidental detail. From `knowledge/adr/ADR-010-AI-Engineering-Assistant.md`:

AI provides recommendations, analysis, documentation and engineering guidance.
All infrastructure execution shall be performed by the Platform Control Plane
after appropriate validation and, where required, human approval.

AI Recommends. Humans Approve. The Platform Executes.

AI is a platform client. It interacts with the platform in exactly the same
way as a self-service portal, a CLI, or a Jira ticket. AI never receives
privileged or direct access to execution engines.

Translated into this lab's current, actual v2 architecture (the same principle, expressed with today's real components rather than ADR-010's original, now-superseded "Platform API"/"Canonical Intent" boxes):



Every one of those five entry points on the top row produces the *same* shape of Nautobot write — this is exactly why adding Jira later (§20) or a bulk LLD loader (§21) requires no change anywhere downstream of Nautobot.

The MCP Server is intentionally the only AI-facing entry point today, but it is designed to remain one of several possible front doors onto the same Nautobot data model — never the only possible one, and never itself an execution engine.

6. Before You Start — Prerequisites and Credentials

Credential	Used by	Where it lives
<code>NAUTOBOT_TOKEN</code>	Generator, MCP Server, write_results, pyATS setup	Nautobot → Profile → API Tokens (or <code>docker/nautobot-isolated/.env</code> 's <code>NAUTOBOT_SUPERUSER_API_TOKEN</code>)
<code>VAULT_ADDR</code> / <code>VAULT_TOKEN</code>	Generator (optional), Terraform, Ansible, pyATS	<code>docker/vault/state/vault-keys.txt</code> ("Initial Root Token")
GitLab trigger token (<code>nautobot-tenant-webhook</code>)	Nautobot's webhook → GitLab pipeline trigger	GitLab → Settings → CI/CD → Pipeline trigger tokens
<code>PIPELINE_STATUS_TOKEN</code>	write_results.py, capture_knowledge.py (read pipeline status)	GitLab Project Access Token, <code>read_api</code> scope
<code>MCP_GITLAB_TOKEN</code>	MCP Server's <code>show_status</code> tool	GitLab Project Access Token, <code>read_api</code> scope, e.g. <code>mcp-server-status-reader</code>
<code>MINIO_ROOT_USER</code> / <code>MINIO_ROOT_PASSWORD</code>	knowledge_capture job	Exported before <code>docker compose up</code> ; must match GitLab's stored CI/CD variables of the same name
APIC credentials (<code>aci_url</code> / <code>aci_username</code> / <code>aci_password</code>)	Terraform, Ansible, pyATS	Vault path <code>secret/lab/platform</code>

Fastest safe path: put every credential in Vault once (`secret/lab/platform`), then every downstream tool (generator, Terraform, Ansible, pyATS) reads it the same way with `--vault-addr/--vault-token` flags or a small loader script — you never hand-type a password into a script twice.

7. Step 1 — Entering Data in Nautobot

Nautobot is where you express *what you want to exist*. Everything downstream (NetAsCode YAML, Terraform, ACI) is derived from what you enter here — never the reverse.

7.1 The object model the generator expects

ACI concept	Nautobot object	Required fields / custom fields
Tenant	<code>tenancy.Tenant</code>	<code>name</code> — must match <code>^[a-z0-9-]+\$</code> (lowercase, digits, hyphens only)
VRF	<code>ipam.VRF</code> (under a Tenant)	<code>name</code> ; optional custom fields <code>aci_ip_data_plane_learning</code> , <code>aci_policy_control_enforcement_direction</code> , <code>aci_policy_control_enforcement_mode</code>
Bridge Domain	<code>ipam.Prefix</code>	<code>description</code> must read exactly <code>ACI Bridge Domain: <bd_name>: <tenant_name></code> ; optional fields <code>aci_bd_mac</code> , <code>aci_bd_arp_flooding</code> , <code>aci_bd_advertise_host_routes</code> , <code>aci_bd_l2_unknown_unicast</code> , <code>aci_bd_l3_unknown_multicast</code> , <code>aci_bd_multi_destination</code> , <code>aci_bd_ep_move_detect_mode</code> , <code>aci_bd_pim</code>
Application Profile + EPG	<code>ipam.VLAN</code>	Custom fields <code>aci_application_profile</code> AND <code>aci_epg_bridge_domain</code> — both must be set or the VLAN is skipped (never silently promoted to ACI config); optional <code>aci_epg_preferred_group_member</code> , <code>aci_epg_contracts</code>
Contracts / Filters	Tenant custom field	<code>aci_contracts</code> (JSON): <code>{"filters": [...], "contracts": [...]}</code>
L3Out (logical only)	Tenant custom field	<code>aci_l3outs</code> (JSON): <code>{"l3outs": [...]}</code>
Fabric/Access Policies	Location custom field	<code>aci_fabric_policies</code> (JSON)

Nautobot's own SSoT namespace convention: the ACI SSoT sync (if enabled) may prefix Tenant names with ACI: — the generator automatically strips this prefix, so type your intent tenant name in Nautobot without the prefix unless your specific instance requires it.

7.2 Worked example — creating one tenant end-to-end via the Nautobot UI

Step 1. Log into Nautobot (<http://localhost:8081> for the isolated lab), go to **Organization** → **Tenants** → **+** **Add**.

Step 2. Name: `example-tenant` (matches `^[a-z0-9-]+$`). Save.

Step 3. Go to **IPAM** → **VRFs** → **+** **Add**. Name: `example-vrf`, Tenant: `example-tenant`. Save.

Step 4. Go to **IPAM** → **Prefixes** → **+** **Add**. Prefix: `10.50.1.0/24`, Tenant: `example-tenant`, VRF: `example-vrf`, Description: `ACI Bridge Domain: example-bd:example-tenant`. Save.

Step 5. Go to **IPAM** → **VLANs** → **+** **Add**. Name: `example-epg`, VID: `100`, Tenant: `example-tenant`. On the custom fields section set `aci_application_profile` = `example-ap` and `aci_epg_bridge_domain` = `example-bd`. Save.

That single tenant now has everything the generator needs to emit a Tenant, a VRF, a Bridge Domain (with a subnet gateway derived from the first host of the prefix), an Application Profile, and an EPG.

7.3 The same thing via the Nautobot REST API

```
export NAUTOBOT_URL=http://localhost:8081
export NAUTOBOT_TOKEN=<your-api-token>

curl -s -X POST "$NAUTOBOT_URL/api/tenancy/tenants/" \
  -H "Authorization: Token $NAUTOBOT_TOKEN" -H "Content-Type: application/json" \
  -d '{"name": "example-tenant", "description": "Guide walkthrough tenant"}'

curl -s -X POST "$NAUTOBOT_URL/api/ipam/vrfs/" \
  -H "Authorization: Token $NAUTOBOT_TOKEN" -H "Content-Type: application/json" \
  -d '{"name": "example-vrf", "tenant": {"name": "example-tenant"}}'

curl -s -X POST "$NAUTOBOT_URL/api/ipam/prefixes/" \
  -H "Authorization: Token $NAUTOBOT_TOKEN" -H "Content-Type: application/json" \
  -d '{"prefix": "10.50.1.0/24", "tenant": {"name": "example-tenant"}, \
    "vrf": {"name": "example-vrf"}, \
    "description": "ACI Bridge Domain: example-bd:example-tenant"}'
```

8. Step 2 — Using the MCP Server Tools (the AI-Agent Path)

Instead of clicking through Nautobot's UI, an AI agent (VS Code Copilot, Claude Desktop) connected to `http://localhost:8071/mcp` can call these tools directly. Every tool writes straight to Nautobot — none of them touch GitLab, Terraform, or ACI directly; they only start the chain described in Section 1.

8.1 Cisco ACI tools (`mcp-server/src/mcp_server/tools/aci.py`)

Tool	Key parameters	What happens
<code>create_tenant</code>	<code>name</code> (must match <code>^[a-z0-9-]+</code>), <code>description</code>	Writes a Nautobot Tenant. If a webhook is configured, a GitLab pipeline is triggered automatically.
<code>create_vrf</code>	<code>tenant</code> , <code>name</code> , <code>description</code>	Writes a VRF under the named tenant.
<code>create_bridge_domain</code>	<code>tenant</code> , <code>vrf</code> , <code>name</code> , <code>gateway_ip</code> (e.g. <code>10.10.10.1/24</code>), <code>description</code>	Writes a Prefix encoding the Bridge Domain per the convention in §3.1.
<code>create_epg</code>	<code>tenant</code> , <code>application_profile</code> , <code>bridge_domain</code> , <code>name</code> , <code>vid</code> (1-4094), <code>description</code>	Writes a VLAN with <code>aci_application_profile / aci_epg_bridge_domain</code> set.
<code>create_contract</code>	<code>tenant</code> , <code>name</code> , <code>filter_name</code> , <code>scope</code> (default <code>context</code>), <code>ether_type</code> (default <code>ip</code>), <code>ip_protocol</code> (default <code>unspecified</code>)	Adds an entry into the Tenant's <code>aci_contracts</code> JSON custom field.
<code>create_l3out</code>	<code>tenant</code> , <code>vrf</code> , <code>name</code> , <code>external_epg_name</code> , <code>subnet</code> (default <code>0.0.0.0/0</code>)	Adds an entry into the Tenant's <code>aci_l3outs</code> JSON custom field. Logical-only — no physical interface binding.

8.2 VXLAN EVPN tools (`mcp-server/src/mcp_server/tools/evpn.py`)

Tool	Key parameters	Notes
<code>create_evpn_tenant</code>	<code>name</code> (without the <code>EVPN:</code> prefix — added automatically)	Writes Tenant as <code>EVPN:<name></code>
<code>create_evpn_vrf</code>	<code>tenant</code> , <code>name</code> , <code>l3_vni</code> (1-16777214)	Sets custom field <code>evpn_l3_vni</code>
<code>create_evpn_bridge_domain</code>	<code>tenant</code> , <code>vrf</code> , <code>name</code> , <code>vlan_id</code> (1-4094), <code>l2_vni</code> (1-16777214)	EVPN Bridge Domains are native VLAN objects, unlike ACI's Prefix encoding

8.3 Generic tool

Tool	Key parameters	Returns
<code>show_status</code>	<code>name</code> (tenant name)	Merges Nautobot's stored <code>validation_status</code> / <code>last_pipeline_id</code> / <code>last_pipeline_url</code> / <code>last_validated_at</code> custom fields with GitLab's <i>live</i> pipeline status for the same tenant.

8.4 Example natural-language session

You: "Create a tenant called example-tenant for the guide walkthrough."
Agent calls: `create_tenant(name="example-tenant", description="Guide walkthrough tenant")`
You: "Add a VRF called example-vrf to it, then a bridge domain example-bd with gateway 10.50.1.1/24."
Agent calls: `create_vrf(tenant="example-tenant", name="example-vrf")` then `create_bridge_domain(tenant="example-tenant", vrf="example-vrf", name="example-bd", gateway_ip="10.50.1.1/24")`
You: "What's the deployment status?"
Agent calls: `show_status(name="example-tenant")` and reports the merged Nautobot/GitLab result back to you.

The MCP Server never runs Terraform or sequences the pipeline itself (ADR-016's "never another orchestration engine" rule). It only writes to Nautobot and reads status back — the actual work is done by the webhook-triggered GitLab pipeline described in the rest of this guide.

9. Step 3 — Generating the NetAsCode YAML from Nautobot

The generator is the one and only translation point between Nautobot and the ACI-specific artifact Terraform actually consumes.

9.1 Manual invocation

```
# Simplest form – token via flag
python3 platform/python/generate_aci.py --url http://localhost:8081 --token <NAUTOBOT_TOKEN>

# Token via environment variable
export NAUTOBOT_TOKEN=<token>
python3 platform/python/generate_aci.py --url http://localhost:8081

# Preview only – print YAML to stdout, write nothing
python3 platform/python/generate_aci.py --url http://localhost:8081 --token <token> --dry-run

# Include ACI system tenants (common/infra/mgmt)
python3 platform/python/generate_aci.py --token <token> --include-system-tenants

# Read credentials from Vault instead of a flag
python3 platform/python/generate_aci.py \
  --vault-addr http://localhost:8200 --vault-token <VAULT_TOKEN>
```

Output is written to `platform/netascode/aci/tenants.yaml` by default (override with `--output <dir>`). The EVPN equivalent is `platform/python/generate_evpn.py`, writing to `platform/netascode/evpn/fabric.yaml` — same flags.

9.2 What the output looks like

```
apic:
  tenants:
    - name: example-tenant
      description: "Guide walkthrough tenant"
      vrfs:
        - name: example-vrf
      bridge_domains:
        - name: example-bd
          vrf: example-vrf
          unicast_routing: true
          subnets:
            - ip: "10.50.1.1/24"
              public: false
              private: true
              shared: false
      application_profiles:
        - name: example-ap
          endpoint_groups:
            - name: example-epg
              bridge_domain: example-bd
```

9.3 Proving determinism (why the pipeline runs the generator twice)

```
python3 platform/python/generate_aci.py --token <token> --output /tmp/run1
python3 platform/python/generate_aci.py --token <token> --output /tmp/run2
diff /tmp/run1/tenants.yaml /tmp/run2/tenants.yaml
# No output = identical = deterministic. This is exactly what the
# "generate_nac" GitLab job does automatically via
# platform/workflows/scripts/check_determinism.sh
```

10. Step 4 — Reviewing Diffs Before Anything Is Applied

Because the generated YAML is a normal Git-tracked file, every change to Nautobot's data becomes a normal, readable `git diff` before it ever reaches Terraform.

10.1 Local diff workflow

```
# After regenerating the YAML locally:
git status platform/netascode/aci/tenants.yaml
git diff platform/netascode/aci/tenants.yaml
```

Example diff you should expect to see after adding the example-tenant walkthrough above:

```
+ - name: example-tenant
+   description: "Guide walkthrough tenant"
+   vrfs:
+     - name: example-vrf
+   bridge_domains:
+     - name: example-bd
+       vrf: example-vrf
+       unicast_routing: true
+       subnets:
+         - ip: "10.50.1.1/24"
+           public: false
+           private: true
+           shared: false
```

10.2 Diff in the GitLab pipeline

The `generate_nac` job commits the regenerated YAML back to the repository automatically (with a `[skip ci]` commit message so it does not re-trigger itself). Review that commit exactly like any other:

```
# In GitLab's UI: Repository → Commits → find the "[skip ci]" commit
# Or from the CLI after a pipeline runs:
git pull
git log -1 -p -- platform/netascode/aci/tenants.yaml
```

Always read this diff before clicking approve on `terraform_apply` (§13.4) — it is the single clearest summary of exactly what is about to change on the real fabric.

11. Step 5 — Schema Validation

Before policy or Terraform ever sees the YAML, a fast structural check runs first.

```
python3 platform/workflows/scripts/validate_nac.py platform/netascode/aci/tenants.yaml
echo "exit code: $?"
# 0 = well-formed (has apic.tenants, every tenant has a name)
# 1 = validation failed
# 2 = usage error (wrong number of arguments)
```

In the GitLab pipeline this is the `validate_nac` job, immediately after `generate_nac` .

12. Step 6 — OPA Policy Enforcement

Policy is a separate question from validation: validation asks "is this technically coherent," OPA asks "is this allowed."

12.1 The actual rule enforced today

```
package platform.cisco_aci

default decision := {"allow": false, "reasons": ["no tenants found in domain_intent"]}

decision := {"allow": true, "reasons": []} if {
  count(invalid_tenant_names) == 0
  count(input.domain_intent.apic.tenants) > 0
}

decision := {"allow": false, "reasons": invalid_tenant_names} if {
  count(invalid_tenant_names) > 0
}

invalid_tenant_names contains msg if {
  some tenant in input.domain_intent.apic.tenants
  not regex.match("[a-z0-9-]+$", tenant.name)
  msg := sprintf("tenant name '%s' does not match required pattern [a-z0-9-]+$", [tenant.name])
}
```

File: docker/platform-api/policy/cisco_aci/tenant_naming.rego. The EVPN equivalent additionally checks VNI range (1-16777214) and VNI uniqueness across tenants/VRFs/bridge-domains.

12.2 Running the policy check yourself

```
# OPA must be reachable (default http://localhost:8181)
export OPA_URL=http://localhost:8181
python3 platform/workflows/scripts/policy_check.py \
  platform/netascode/aci/tenants.yaml cisco_aci
echo "exit code: $?"
# 0 = allowed
# 1 = denied, OR opa unreachable (fails CLOSED, never open, per ADR-014)
# 2 = usage error
```

12.3 Starting OPA if it is not already running

```
docker run -d --name opa -p 8181:8181 \
  -v "$(pwd)/docker/platform-api/policy:/policy:ro" \
  openpolicyagent/opa:latest run --server --addr=0.0.0.0:8181 /policy
```

If OPA cannot be reached, `policy_check.py` treats that as a **denial**, not a pass — the pipeline will not silently skip policy just because OPA is down.

13. Step 7 — Working With GitLab

13.1 Create the project (one-time, per GitLab instance)

Step 1. Open `http://localhost:8929`, sign in as root (initial password: `docker exec <gitlab-container> cat /etc/gitlab/initial_root_password`, valid 24 hours).

Step 2. Create a new project, e.g. `root/nautobot-infra-automation`.

Step 3. Push this repository's contents (`pipelines/`, `.gitlab-ci.yml`, `platform/`) to that project's main branch.

Step 4. Register a Runner: **Settings** → **CI/CD** → **Runners**, copy the registration token, then run the `gitlab-runner` container pointed at this GitLab instance (see `docker/other-containers/gitlab-runner/docker-compose.yml`).

13.2 Set the required CI/CD variables

Settings → **CI/CD** → **Variables**, add:

Variable	Purpose
<code>NAUTOBOT_TOKEN</code>	Used by <code>generate_nac</code> , <code>write_results</code>
<code>PIPELINE_STATUS_TOKEN</code>	Used by <code>write_results</code> / <code>knowledge_capture</code> to read pipeline status via <code>PRIVATE-TOKEN</code> (not the ambient CI job token)
<code>MINIO_ROOT_USER</code> / <code>MINIO_ROOT_PASSWORD</code>	Used by <code>knowledge_capture</code> — must match the running MinIO container's real credentials
<code>GIT_PUSH_TOKEN</code>	Lets <code>generate_nac</code> commit the regenerated YAML back to the repo

13.3 Create the webhook trigger token

```
# Settings -> CI/CD -> Pipeline trigger tokens -> Create trigger token
# Name it e.g. "nautobot-tenant-webhook", copy the token value.

# Manual test of the exact same call Nautobot's webhook makes:
curl -X POST "http://localhost:8929/api/v4/projects/1/trigger/pipeline" \
  -F "token=<trigger-token>" -F "ref=main"
```

In Nautobot: **Admin** → **Extensibility** → **Webhooks** → **Add**. Content type `tenancy.tenant`, Type create = true, URL = the GitLab trigger endpoint above, HTTP method POST, Body content type `application/x-www-form-urlencoded`. Once saved, every new Tenant automatically fires a pipeline with `source: "trigger"` in the GitLab UI (a normal `git push` instead shows `source: "push"`).

13.4 The manual approval gate

`terraform_apply` is marked `when: manual` in the pipeline — it will not run until a human clicks "Play" in the GitLab pipeline view. In this lab's GitLab CE (no Premium Protected Environments), the substitute control is the `main` branch's protected-branch rule: only Maintainer-level users can trigger the manual job.

Step 1. Open the pipeline in **CI/CD** → **Pipelines**.

Step 2. Confirm `generate_nac`, `validate_nac`, `policy_check`, and `terraform_plan` are all green.

Step 3. Open `terraform_plan`'s job log and read the plan output — this is your last chance to see exactly what Terraform intends to create/change/destroy.

Step 4. If it looks correct, click the manual ► **terraform_apply** job to let it run.

13.5 Monitoring a pipeline

```
# Web UI
http://localhost:8929/root/nautobot-infra-automation/-/pipelines

# API
curl -s -H "PRIVATE-TOKEN: <token>" \
  "http://localhost:8929/api/v4/projects/1/pipelines" | python3 -m json.tool

# Via MCP (from an AI agent, or by calling the tool directly)
show_status(name="example-tenant")
```

14. Step 8 — Terraform Plan and Apply Against Cisco ACI

14.1 Loading credentials from Vault

```
export VAULT_ADDR=http://localhost:8200
export VAULT_TOKEN=$(grep "Initial Root Token" docker/vault/state/vault-keys.txt | awk '{print $NF}')
source platform/terraform/aci/scripts/load-vault-creds.sh
# This exports TF_VAR_aci_url, TF_VAR_aci_username, TF_VAR_aci_password, TF_VAR_aci_insecure
```

14.2 Plan, review, apply

```
cd platform/terraform/aci
terraform init -input=false
terraform plan -input=false -lock-timeout=60s -out=tfplan
# READ THE PLAN OUTPUT NOW — this is the same review point as GitLab's manual gate
terraform apply -input=false -auto-approve -lock-timeout=60s tfplan
cd -
```

Required variables (from platform/terraform/aci/variables.tf): `aci_url` (e.g. `https://172.30.46.103`), `aci_username`, `aci_password`, `aci_insecure` (bool). Provider: `CiscoDevNet/aci ~> 2.0`. Terraform version pinned in CI: `1.9.8`.

Never run `terraform apply` without first running `terraform plan` and reading its output — this is the single most important safety habit in this entire lab. The GitLab pipeline enforces this by construction (separate `plan` and `apply` stages with a manual gate in between); do the same when running locally.

15. Step 9 — Ansible Day-2 Configuration

```
cd platform/ansible/aci
ansible-playbook -i inventory/hosts.yml playbooks/verify-tenants.yml
ansible-playbook -i inventory/hosts.yml playbooks/day2-epg.yml
cd -
```

`verify-tenants.yml` loads the generated YAML, queries ACI via `cisco.aci.aci_tenant` (state: query), and asserts every expected tenant is present — this is a second, Ansible-based sanity check that runs before the independent pyATS verification. `day2-epg.yml` is a worked example that ensures an Application Profile and EPG exist and are bound to a Bridge Domain, using `cisco.aci.aci_ap` and `cisco.aci.aci_epg`.

Inventory (`platform/ansible/aci/inventory/hosts.yml`) uses `ansible_connection: local` — there is no SSH; the `cisco.aci` collection talks to the APIC's REST API directly using credentials from `group_vars/aci.yml` (populated from Vault at runtime, not version-controlled).

16. Step 10 — pyATS Independent Verification

pyATS is deliberately independent of Terraform and Ansible — it proves the fabric matches intent using nothing but a read-only REST query, so a bug in the Terraform module cannot also hide the verification failure.

```
export VAULT_ADDR=http://localhost:8200
export VAULT_TOKEN=<token>
source tests/pyats/aci/scripts/load-vault-env.sh
# exports ACI_HOST, ACI_USERNAME, ACI_PASSWORD from secret/lab/platform

pyats run job tests/pyats/aci/job.py \
  --testbed-file tests/pyats/aci/testbed.yml --no-mail
```

What it actually checks (tests/pyats/aci/test_aci_tenants.py): loads the expected tenant list from platform/netascode/aci/tenants.yaml, queries the live APIC's `api/class/fvTenant.json`, and fails the test if any expected tenant is missing from the response. The testbed's device `os` must be `apic` (not `aci`) to match pyATS' generic REST connector.

Results are archived as a ZIP under `pyats-results/` when run inside the GitLab pipeline's `pyats_verify` job (an artifact you can download from the job page even on failure, since the job's artifacts use `when: always`).

17. Step 11 — MinIO Knowledge Capture and Audit Trail

The final pipeline stage writes one durable JSONL record per run, tying the whole chain together — intent, policy decision, execution, and verification — in one place.

```
export MINIO_ENDPOINT=http://localhost:9000
export MINIO_ROOT_USER=<user>
export MINIO_ROOT_PASSWORD=<password>

python3 platform/workflows/scripts/capture_knowledge.py \
  knowledge_capture/aci-deployments.jsonl \
  knowledge-capture \
  aci/deployments.jsonl
```

Record shape (one JSON object per line, appended — never overwritten):

```
{
  "captured_at": "2026-09-01T09:14:22.512Z",
  "pipeline_status": "success",
  "pipeline_id": "42",
  "pipeline_url": "http://localhost:8929/root/nautobot-infra-automation/-/pipelines/42",
  "commit_sha": "94586ac...",
  "job_id": "118",
  "operator": "root",
  "domain_id": "cisco_aci",
  "netascode_yaml": "platform/netascode/aci/tenants.yaml",
  "environment": "lab"
}
```

17.1 Reading the log back

```
# Via the MinIO console
http://localhost:9091

# Via the AWS/S3 CLI (MinIO is S3-compatible)
aws --endpoint-url http://localhost:9000 s3 cp \
  s3://knowledge-capture/aci/deployments.jsonl - | tail -5
```

17.2 Starting MinIO if it is not already running

```
export MINIO_ROOT_USER=miniolabadmin
export MINIO_ROOT_PASSWORD=<choose-a-strong-password>
docker compose -f docker/other-containers/minio/docker-compose.yml up -d
```

If you recreate the MinIO container, its `MINIO_ROOT_USER`/`MINIO_ROOT_PASSWORD` must match whatever GitLab's stored CI/CD variables already expect — a mismatch here silently breaks only the `knowledge_capture` job while everything else keeps working, which makes it easy to miss.

18. End-to-End Worked Example

Putting every step above together for one concrete change, from a blank Nautobot to a verified, recorded ACI deployment.

1. **Nautobot:** create Tenant `example-tenant`, VRF `example-vrf`, Prefix `10.50.1.0/24` described `ACI Bridge Domain: example-bd:example-tenant`, VLAN `example-epg` (vid 100, `aci_application_profile=example-ap`, `aci_epg_bridge_domain=example-bd`) — see §7.2.
2. **Trigger:** either the Nautobot webhook fires automatically on the Tenant's creation, or push to `main` manually.
3. **generate_nac:** pipeline job regenerates `tenants.yaml`, proves determinism, commits it — review the diff (§6).
4. **validate_nac:** schema check passes (exit 0).
5. **policy_check:** OPA confirms `example-tenant` matches `^[a-z0-9-]+$` — `allow: true`.
6. **terraform_plan:** produces a plan showing new Tenant/VRF/BD/AP/EPG resources — read it in the job log.
7. **Manual approval:** a Maintainer clicks ► on `terraform_apply` after reviewing the plan.
8. **terraform_apply:** applies to the real APIC.
9. **ansible_configure:** confirms the tenant is queryable in ACI, applies any Day-2 EPG example.
10. **pyats_verify:** independently re-queries `fvTenant.json` and confirms `example-tenant` exists.
11. **write_results:** Nautobot's `example-tenant` now shows `validation_status: stable`, `last_pipeline_id`, `last_validated_at`.
12. **knowledge_capture:** one new line appended to `s3://knowledge-capture/aci/deployments.jsonl`.
13. **Confirm from anywhere:** `show_status(name="example-tenant")` via MCP returns the same merged Nautobot + live GitLab status.

19. Command Cheat-Sheet

Action	Command
Generate YAML (dry-run)	<code>python3 platform/python/generate_aci.py --token <T> --dry-run</code>
Generate YAML (write)	<code>python3 platform/python/generate_aci.py --token <T></code>
Validate schema	<code>python3 platform/workflows/scripts/validate_nac.py platform/netascode/aci/tenants.yaml</code>
Policy check	<code>python3 platform/workflows/scripts/policy_check.py platform/netascode/aci/tenants.yaml cisco_aci</code>
Diff before apply	<code>git diff platform/netascode/aci/tenants.yaml</code>
Terraform plan	<code>terraform -chdir=platform/terraform/aci plan -out=tfplan</code>
Terraform apply	<code>terraform -chdir=platform/terraform/aci apply tfplan</code>
Ansible verify	<code>ansible-playbook -i platform/ansible/aci/inventory/hosts.yml platform/ansible/aci/playbooks/verify-tenants.yml</code>
pyATS verify	<code>pyats run job tests/pyats/aci/job.py --testbed-file tests/pyats/aci/testbed.yml -- no-mail</code>
Trigger pipeline manually	<code>curl -X POST http://localhost:8929/api/v4/projects/1/trigger/pipeline -F token=<T> -F ref=main</code>
Check pipeline status	<code>curl -H "PRIVATE-TOKEN: <T>" http://localhost:8929/api/v4/projects/1/pipelines</code>
Read knowledge capture log	<code>aws --endpoint-url http://localhost:9000 s3 cp s3://knowledge- capture/aci/deployments.jsonl -</code>

20. Future Integration — Jira as a Ticketing Front-Door for Day-2 Operations

Status: not implemented in this repository today. Everything below is a recommended design, built directly on principles this platform already states for itself (ADR-010's "many entry points, one execution path" and ADR-019's Three Truths model) — it is not a description of existing code.

ADR-010 already lists a Jira ticket as one of several equally valid platform clients: *"It interacts with the platform in exactly the same way as a self-service portal, a CLI or a Jira ticket."* The recommended pattern keeps that promise literally true.

20.1 Recommended architecture

```
Jira issue (type: "Network Change Request")
  custom fields: tenant, vrf, bridge_domain/gateway, ep, ticket_id
  |
  v  transitions to "Approved"
Jira Automation rule -> webhook
  |
  v
Adapter (either: a new Nautobot Job, OR a new MCP business-operation
  tool per ADR-019's forward-compatibility note, e.g. a future
  create_day2_change_request tool)
  |
  v
Nautobot write — SAME object model as §7.1, PLUS a new
  custom field/tag: jira_ticket_id = "NETOPS-1234"
  |
  v
[ everything from §3 onward is completely unchanged ]
  |
  v
GitLab pipeline -> OPA -> Terraform/Ansible -> ACI -> pyATS -> Nautobot/MinIO
```

20.2 Why the ticket ID matters architecturally

ADR-019's Three Truths model names **Business Intent** ("why a change was requested") as a truth the platform does not yet capture explicitly — today it is implicit in commit messages and JSONL records only. Storing the Jira ticket ID as a Nautobot custom field on the affected object(s) is the simplest concrete step toward making Business Intent explicit, without inventing a new schema: every downstream artifact (the generated YAML, the Terraform plan, the knowledge-capture record) can now be traced back to the ticket that requested it.

20.3 What would need to change

- A new Nautobot custom field, e.g. `jira_ticket_id`, on Tenant (and optionally VRF/Prefix/VLAN for finer-grained traceability).
- One adapter component (Nautobot Job or MCP tool) that validates the Jira webhook payload and performs the Nautobot write — reusing the exact object model in §7.1, so the generator, OPA, Terraform, and Ansible never need to change.
- An optional one-line addition to `capture_knowledge.py`'s record shape (§17) — a `ticket_id` field — so the audit trail closes the loop back to the originating request.

Build the adapter as a Nautobot Job first (Nautobot already runs one custom Job today — the ACI SSoT sync — so the extensibility mechanism is proven in this platform) before considering a dedicated MCP tool; only add the MCP tool once an AI agent, not just Jira, also needs to originate the same kind of change.

21. Instructing Nautobot via a Low-Level Design (LLD) Document

Status: not implemented in this repository today. No "Golden Config" app, "Design Builder" app, or bulk-LLD-loading Job exists in this platform currently (confirmed by searching the codebase) — the pattern below is the recommended way to build one, consistent with how Nautobot is already extended here.

Yes, this is a good idea — but feed the LLD to Nautobot, never directly to ACI. Nautobot must remain the single ingestion point regardless of where intent originates (UI click, MCP tool, Jira ticket, or a bulk LLD file) — this is exactly what keeps the generator, policy, and pipeline unchanged no matter which front door was used, per §5's diagram.

21.1 Recommended pattern

1. **Express the LLD in a structured, versioned format** (YAML, CSV, or Excel) whose fields map directly onto the object model and custom-field conventions already in §7.1 — do not invent a second, competing schema.
2. **Write a custom Nautobot Job** (Python, using Nautobot's own `nautobot.apps.jobs` extensibility — the same mechanism this platform already uses for the Cisco ACI SSOT sync job) that: accepts the LLD file as a Job input (Nautobot Jobs support `FileVar` / `StringVar` inputs), parses it, and creates/updates Tenant/VRF/Prefix/VLAN objects and their custom fields via Nautobot's ORM.
3. **Give the Job a dry-run mode** that reports what it would create/change without writing anything — mirroring the generator's own `--dry-run` convention (§9.1) so the same safety habit applies at ingestion, not just at generation.
4. **Run the Job** via Nautobot's Jobs UI (`http://localhost:8081/extras/jobs/`) or its REST API: `POST /api/extras/jobs/{job_id}/run/`.
5. **Everything from that point is unchanged** — the generator reads Nautobot exactly as it always does, and the same GitLab pipeline runs.

21.2 Illustrative Job skeleton (not existing code — a starting pattern)

```
from nautobot.apps.jobs import Job, FileVar, register_jobs

class ImportLowLevelDesign(Job):
    lld_file = FileVar(description="YAML low-level design document")

    class Meta:
        name = "Import Low-Level Design"
        description = "Bulk-loads Tenants/VRFs/Bridge Domains/EPGs from an LLD file"

    def run(self, lld_file):
        design = yaml.safe_load(lld_file.read())
        for tenant_def in design["tenants"]:
            tenant, _ = Tenant.objects.get_or_create(name=tenant_def["name"])
            # ... create VRFs/Prefixes/VLANs using the same custom-field
            # conventions the generator already expects (see 7.1) ...
        self.logger.info("Import complete: %s tenant(s) processed", len(design["tenants"]))

register_jobs(ImportLowLevelDesign)
```


This is a plain Nautobot Job doing a bulk write — not "Golden Config" and not "Design Builder" (neither is used anywhere in this platform today). Keeping it this simple means it stays a thin ingestion layer, not a second competing source of truth.

22. Future Enhancement — A LangGraph Reasoning Agent

Status: designed, not built. This section is grounded entirely in `knowledge/ai/Future-AI-Integration-Design.md` and `knowledge/adr/ADR-019-Three-Truths-Principle.md` — both explicitly describe this as a future contract, not existing code: *"Explicitly out of scope until a later phase, even after this design is agreed: any Vector DB, embedding pipeline, or actual LangGraph graph/agent code. This document is the contract those future components will be built against — it does not build them."*

22.1 What LangGraph would add

Quoting the design document directly: LangGraph is *"the reasoning layer that combines"* three sources the MCP Server does not itself combine today:

1. **Semantic retrieval** from this knowledge base — architecture context, past decisions, prior incident write-ups.
2. **Structured retrieval** from Nautobot — live current state.
3. **Execution history** from GitLab (pipeline/job status) and the Knowledge Capture JSONL records (§17).

22.2 Where it would sit

```

Engineer: "Provision a new customer zone spanning ACI and EVPN,
          apply the standard security profile, and confirm compliance."
          |
          v
LangGraph reasoning agent
  1. retrieve relevant architecture/policy context from knowledge/
  2. query Nautobot for current state (avoid duplicate/conflicting objects)
  3. check GitLab + knowledge-capture history for prior similar changes
  4. decide a sequence of MCP tool calls
          |
          v
MCP Server (same tools this guide already documents — §8)
  create_tenant -> create_vrf -> create_bridge_domain      (ACI)
  create_evpn_tenant -> create_evpn_vrf -> create_evpn_bridge_domain (EVPN)
          |
          v
Nautobot -> GitLab pipeline -> Terraform/Ansible -> ACI/EVPN fabric (unchanged, §3)

```

For a single-domain, single-step request ("create a tenant"), LangGraph adds no value — the existing direct MCP tool call already handles it. It becomes useful specifically for **multi-domain, multi-step business operations** that today would otherwise require a human to manually sequence several separate tool calls.

22.3 What this design deliberately preserves

- LangGraph never bypasses the MCP Server — it calls the same tools any AI agent already calls.
- The MCP Server still never orchestrates the pipeline itself (ADR-016) — LangGraph does the multi-step *reasoning*, GitLab still does the multi-step *execution*.
- No new write path to Nautobot is invented — every write is still one of the existing per-tool schemas (ADR-018), never a new shared "intent" object.

22.4 Recommended build order

Consistent with this platform's own proven discipline ("prove the mechanism narrow, widen later" — the same approach used to take ACI's pipeline mechanism and prove it also works for EVPN, per ADR-021):

1. **First**, add real business-operation tools (e.g. a future `provision_customer_zone`) only once at least two matching single-domain operations already exist and are proven — today that means after both `create_tenant` (ACI) and `create_evpn_tenant` (EVPN) are in real use.
2. **Then**, add semantic retrieval over knowledge/ only once a real multi-step reasoning task exists that actually needs it — not speculatively ahead of time.
3. **Only then**, introduce LangGraph itself as the agent deciding tool-call order. Building the graph before there is a real multi-step business operation to justify it would repeat the exact "custom orchestration nobody needed yet" mistake ADR-016 already identified and corrected once in this platform's own history (Platform v1's custom Python orchestration, replaced by GitLab-native capabilities).

23. Troubleshooting and Operational Hazards

Symptom	Likely cause / fix
<code>policy_check</code> fails on an unrelated tenant	The generator bundles <i>all</i> Nautobot tenants into one YAML file — one bad tenant/VRF anywhere fails the whole pipeline. Check the job's OPA error message before assuming a platform bug.
OPA unreachable	<code>policy_check.py</code> fails closed (denies), by design (ADR-014) — this is correct behavior, not a bug. Start/restart the <code>opa</code> container.
<code>terraform_apply</code> never becomes runnable	Confirm your GitLab user has Maintainer access on the protected <code>main</code> branch — Developer-level users get <code>403</code> on the manual job, by design.
<code>knowledge_capture</code> fails silently while everything else is green	Check that <code>MINIO_ROOT_USER</code> / <code>MINIO_ROOT_PASSWORD</code> GitLab CI variables match the actually-running MinIO container's credentials.
EPG never appears in generated YAML	Both <code>aci_application_profile</code> AND <code>aci_epg_bridge_domain</code> custom fields must be set on the VLAN — either one missing means it is treated as an ordinary, non-ACI VLAN.
pyATS testbed connection fails	Confirm the device's <code>os:</code> is <code>apic</code> , not <code>aci</code> — this is the exact abstraction token pyATS' REST connector expects.
Never run <code>docker compose down</code> from a subdirectory	Every service shares the project name <code>infra-automation-lab</code> — a scoped <code>down</code> reconciles (and can stop) the entire lab. Use <code>docker compose up -d --no-deps <service></code> from the root <code>docker/</code> directory instead.

Every command, tool name, parameter, and file path in this guide was verified directly against the source in this repository (not inferred) as of 1 September 2026. If a script's behavior changes, re-check the referenced file before relying on this guide.